

AWS GenAI Developer — Domain 1 (Hinglish)

Solution Design · Tasks 1.1 se 1.6 · Chat wali explanations, document form mein

Yeh document wahi explanations hai jo humne baat-cheet mein cover ki. **Orange boxes** mein wo baatein hain jo sabse zaroori hain, **green boxes** mein analogies. Technical terms English mein hi rakhe hain — exam mein wahi shabd dikhenge.

Task 1.1 — Architecture aur PoC

Use case

Ek multinational bank — 15 countries, 8 languages. Customer sawaal poochta hai, AI jawaab deta hai. Par bank ke apne documents se, aur regulation ke andar rehte hue.

Ek request ka poora safar

Marie France mein hai. Wo French mein poochti hai: "agar main FD tod doon toh kitna charge lagega?"

Step 1 — DNS

Marie ke phone ko sirf **naam** pata hai — `abc.com`. Computer ko **number** chahiye. DNS phone directory hai — naam daalo, number milta hai.

Khaas baat: Global Accelerator ke saath poori duniya mein sabko **ek hi do IPs** milti hain.

Step 2 — Anycast

Analogy: ek coffee chain hai jiski har branch ka pata ek hi hai. Aap chitthi bhejte ho, postman use sabse paas wali branch mein pahuncha deta hai. AWS wahi karta hai — do IPs sainkdon edge locations se announce hoti hain.

Yahan sabse badi galatfehmi: anycast Marie ko "dhoondhta" nahi. Koi lookup nahi, koi geolocation nahi. **BGP routing** hai — har router apne padosiyon se sunta hai ki us IP tak ka raasta kitna lamba hai, aur sabse chhota chun leta hai. Router ne Marie ki location dekhi hi nahi. Jaise paani apne aap dhalaan par behta hai.

Aur router ko **poora raasta nahi pata** — sirf agla ek kadam. Postman jaisa: Mumbai wale ko sirf "Delhi bhejo" pata hai, Delhi ki gali Delhi wale ka kaam hai.

Step 3 — Global Accelerator

Ab GA shuru hota hai — Paris edge par. Wo dekhta hai kaunsa region healthy hai aur allowed hai, phir AWS ke private backbone se wahan bhejta hai.

Public internet = aam sadak (traffic, signal, jaam)
AWS backbone = private expressway

GA ka poora fayda yahi hai — Marie ko ekdum shuru mein, Paris mein, expressway par chadha dena.

Do IPs kyun? Sirf backup ke liye — alag network zones se aati hain. **Region se koi lena-dena nahi.** 1 region ho ya 20, IPs hamesha 2 hi rahengi. Ek ghar, do gate — matlab do ghar nahi.

Aur IP "down" nahi hoti, kyunki IP koi machine nahi hai — wo 200 jagah se announce ho rahi hai. Ek edge par problem ho toh BGP khud us edge ko hata deta hai.

Ek problem jo diagram nahi batata: GA mein "EU users sirf EU mein" wali setting **hai hi nahi**. Wo sirf nearest healthy dekhta hai. Data residency ke liye traffic dial 0% karna padta hai, ya alag accelerators banane padte hain.

Step 4 — AppSync

Marie ab Ireland pahunch gayi. AppSync **front desk** hai. Teen kaam: sawaal lena, token check karna, Lambda ko kaam dena.

AppSync ka asli fayda — subscriptions. Claude ka jawaab 5 second leta hai. REST mein Marie 5 second spinner dekhti. AppSync mein ek **permanent line** jud jaati hai (WebSocket) — jaise phone call chalu rehna. Lambda jaise-jaise words banata hai, wo Marie ki screen par aate rehte hain.

Ek pushback jo yaad rakhne layak hai: AppSync yahan sabse achha choice nahi hai. Chat assistant ek hi mutation hai — GraphQL ka asli fayda (client apni marzi ka data maange) yahan lagta hi nahi. Timeout bhi ~30s hai, API Gateway jitna. **Sirf subscriptions aur field-level auth ki wajah se defensible hai.**

Step 5 — Cognito

Order galat samajhna aam hai: Cognito AppSync se "pehle" nahi hai. **Cognito AppSync ke andar hai.** Marie ka request seedha AppSync par aata hai; AppSync khud token check karta hai. Cognito ko phone nahi karta.

Analogy: आपको joining के दिन office ID card मिला (Cognito). अब रोज़ guard सिर्फ card देखता है — HR को call नहीं करता.

Token से पता चलता है: कौन है, कौनसे group में है, कौंसी jurisdiction है. **Jurisdiction Marie ने type नहीं की** — token से आयी है, इसीलिए ज़हूँ नहीं बोल सकती.

Step 6-7 — Lambda और Knowledge Base

सबसे बड़ी गलती जो लोग करते हैं: "model knowledge base में search करेगा" — **नहीं. Model कभी कुछ search नहीं करता.**

Lambda पहले KB से pages निकालता है, फिर prompt में चिपका कर model को देता है.

Analogy: आप एक smart wakeel को बुलाते हैं जैसे आपके case का कुछ नहीं पता. आप पहले almari से 5 कागज़ निकालते हैं, उसके सामने रखते हैं, फिर पूछते हैं. **Wakeel almari खुद नहीं खोलता.**

Knowledge Base के अंदर — search कैसे होता है

Problem: Marie French में पूछती है, document English में है. एक भी word match नहीं करता.

Solution — embedding. हर sentence को numbers की line में बदल देते हैं:

"जल्दी पैसे निकालना"	-> [0.2, 0.8, 0.1, ...]	
"premature withdrawal"	-> [0.2, 0.8, 0.1, ...]	लगभग same!
"credit card apply"	-> [0.9, 0.1, 0.7, ...]	बहुत अलग

Analogy: एक बड़े map पर हर sentence को जगह मिलती है. मिले-जुलते matlab वाले पास-पास बैठते हैं — **चाहे बhashा कौी भी हो.**

Metadata filter search से पहले लगता है: `jurisdiction = FR`. US के documents **देखें ही नहीं जाते**. यही वो mechanism है जिसे bank का compliance चलाता है.

Do चीज़ें जो diagram में नहीं हैं:

- 1. **Vector store** — Knowledge Base खुद vectors store नहीं करता. OpenSearch Serverless या Aurora pgvector चाहिए, जो आप बनाते हैं और उसका पैसा लगता है.
- 2. **Guardrails** — regulated bank के लिये सबसे बड़ी कमी. PII masking, denied topics, और contextual grounding (hallucination check).

Ingestion — documents अंदर कैसे आते हैं

यह Marie के सवाल के वक़्त नहीं होता. **पहले से, offline** हो चुका होता है.

- 1. Data source connect (S3, SharePoint, Confluence)
- 2. Text निकालो (formatting, images हट जाते हैं)
- 3. CHUNKING - तुकड़ों में काटो (~300-500 words, 10% overlap)
- 4. हर chunk का अपना EMBEDDING (एक-एक करके)
- 5. Store: text + numbers + METADATA साथ

Metadata ingestion के वक़्त daalna padta है. बाद में नहीं daal सकते — पूरा दोबारा ingest करना पड़ेगा.

और **embedding model दोनों तरफ़ same होना चाहिए** — query पर भी, documents पर भी. Alag model = alag vector space = कुछ match ही नहीं करेगा.

Chunking की सच्चाई: कोई formula नहीं है, trial and error है. Chunk छोटा → precision अच्छी, context तोटा. Bड़ा → context पूरा, शोर ज्यादा.

Prompt और Guardrails

Lambda के पास तीन चीज़ें हैं: template (Prompt Management से), chunks (KB से), और सवाल. सब जोड़कर model को भेज दिया. **Jo prompt में है, वही model की पूरी दुनिया है.**

Prompt Management का फायदा: compliance team disclaimer बदलना चाहती है? Version 8 बना देंगी. **Code में कुछ नहीं बदला, deploy कुछ नहीं हुआ.**

Guardrails दो जगह लगते हैं — input पर (prompt injection, PII) और output पर (denied topics, PII masking, contextual grounding).

Contextual grounding: guardrail check करता है कि जो model ने कहा, वो chunks में सच में लिखा था या नहीं. Model ने "penalty 2%" कहा पर chunks में 2% कहिन नहीं? **Model ने बना दिया** — guardrail रोक देता है. यह hallucination का सीधा इलाज है.

EventBridge और monitoring

Lambda एक note लिखता है और **bhool जाता है** — जवाब का इंतेज़ार नहीं करता. Marie का जवाब जा चुका है.

Fayda: अगर Lambda ने सीधा CRM call किया होता और CRM down होता — **CRM का outage customer का outage बन जाता.** EventBridge में event रुका रहेगा, retry होगा, ना चला तो DLQ. Marie को पता भी नहीं चलेगा.

Service	Sawaal
CloudWatch	Kitna? Kitni der?
X-Ray	Kahan atka?
SageMaker Pipelines	Jawaab abhi bhi sahi hai?

Task 1.2 — Model Selection aur Lifecycle

Use case aur ek zaroori farq

Healthcare — 50,000 medical documents roz, HIPAA compliance.

	Task 1.1 (Bank)	Task 1.2 (Healthcare)
Model kaise sudhara	RAG — documents prompt mein	Fine-tuning — model khud badla
Kyun	Facts roz badalte hain	Medical bhasha seekhni hai

Model selection — sahi tarika

Galat: blog padho, "Claude sabse achha hai" dekh kar chun lo. **Sahi:** apne data par test karo, numbers dekho, phir chuno.

1. CRITERIA pehle likho (warna baad mein ghuma doge)
 2. Dataset banao (100-300 examples, representative)
 3. Bedrock Model Evaluation chalao
 4. Latency aur cost ALAG se measure karo
 5. Limitations dhoondho
 6. Har kaam ke liye alag model chuno

Evaluation ke teen tarike: Automatic (tez, sasta), LLM-as-judge (custom metrics, medical ke liye behtar), Human (slow, mehenga, par clinical sign-off ke liye zaroori).

Latency aur cost Model Evaluation nahi deta. Alag se script chalao. Aur yahin faisla aksar palat jaata hai — sabse accurate model 5x mehenga nikal sakta hai.

Limitations kaise pakadte hain

Aapne poocha tha ki yeh manual hota hai kya. Jawaab: **teeno — automatic, manual, aur production se.**

common_disease:	96%
rare_disease:	71% <- yeh 89% average mein CHHUPA tha
handwritten:	64%

Slicing automatic hai — par tagging manual. **Failure padhna** poora manual hai (aur medical mein doctor chahiye). **Adversarial testing** — "patient denies chest pain" par model kya karta hai. **Production drift** baad mein Model Monitor pakadta hai.

Model Evaluation aapko **score** deta hai. **Limitation aapko khud dhoondhni padti hai.**

Abstraction layer

Problem: Claude aur Titan ka API alag hai. Seedha call karoge toh code mein 20 jagah `if model == ...` aa jaayega. Naya model = 20 jagah change. **Yeh vendor lock-in hai** — badalne ki keemat itni ki aap badloge hi nahi.

API Gateway	-> sabke liye ek interface
Router Lambda	-> kaunsa model? (AppConfig se pooche)
Adapter Lambda->	model ki bhasha mein translate
Bedrock	

AppConfig ek jagah nahi hai — wo ek settings file hai. Router aur Adapter usmein "nahi" hain. Router bas usse ek value **padh leta hai.**

Analogy: Router ek driver hai. AppConfig ek parchi hai jispar likha hai "aaj Claude ke ghar jaana hai". Driver parchi padhta hai, phir gaadi chalata hai. **Driver parchi ke andar nahi baitha.**

Adapter kya karta hai

Sirf Bedrock call nahi — **teen kaam, aur asli kaam translation hai:**

1. AAGE ka translation (aapka format -> model ka format)
 2. Bedrock call (sabse chhota hissa)
 3. PEECHE ka translation (model ka jawaab -> aapka format)

Analogy: aap Hindi bolte ho, doctor French. Translator aapki baat French mein kehta hai, doctor ka jawaab Hindi mein wapas laata hai. **Dono taraf kaam karta hai.**

Resilience — teen tarah ke failures

Failure	Ilaaj
Transient (ek baar ki dikkat)	Retry with exponential backoff + jitter
Sustained (model hi down)	Circuit breaker — koshish hi band
Degraded (chal raha hai, jawaab kharab)	CloudWatch custom metrics

Circuit breaker = bijli ka fuse. Short circuit ho toh fuse ud jaata hai, taaki poora ghar na jale. Open state mein aap **koshish hi nahi karte** — 30 second wait karke fail hone se achha turant fallback.

Degraded sabse chalak failure hai: latency theek, error rate zero, aur jawaab kharab. Normal monitoring ise pakad hi nahi sakti.

Graceful degradation: "kuch nahi" jawaab galat hai, "thoda kam" sahi hai.

Analogy: hospital mein bijli gayi, generator chalu ho gaya. Poori building nahi, par ICU aur emergency chal rahe hain. Sab band karne se behtar.

Fine-tuning aur LoRA

Rule: naya **gyaan** chahiye → RAG. Naya **tarika/format** chahiye → fine-tuning.

Aur ek zaroori chetavni: **fine-tuning knowledge cutoff theek nahi karta.** Nayi guidelines uske liye RAG chahiye. Asli systems mein dono hote hain — format model se, facts retrieval se.

LoRA: poore model ko mat chhedo. Ek chhoti parat jodo aur sirf usko train karo (~1%).

Analogy: ek doctor jo general medicine jaanta hai. Aap uska dimaag nahi badal rahe — bas ek specialty ki chhoti training de rahe ho. **Base knowledge wahi rehti hai.** Aur ek base par kai adapters lag sakte hain — radiology ka alag, pathology ka alag.

Lifecycle

```
Fine-tuned model
-> Model Registry (version + lineage)
-> CLINICAL VALIDATION (insaan approve kare)
-> CodePipeline (accuracy / HIPAA / EHR tests)
-> Blue-Green ya Canary deploy
-> Model Monitor (drift dekhta rahe)
-> drift mila -> dobara train
-> purana retire, KNOWLEDGE TRANSFER
```

Clinical validation jaan-boojh kar manual hai. Automation ka matlab insaan hatana nahi.

Aur **drift:** model deploy ke baad dheere-dheere kharab hota hai, code badle bina — kyunki **duniya badal jaati hai.** Nayi terminology, naye document formats. CloudWatch batata hai "system chal raha hai"; Model Monitor batata hai "jawaab abhi bhi sahi hai".

Task 1.3 — Data Preparation

Use case

Insurance claims — 50,000/month. Aur yahan **naya cheez multimodal hai**: claim forms (tabular), damage photos (image), call recordings (audio), historical data.

Chaaron ko alag process karo, phir **jodo**, phir model ko do. Yahi Task 1.3 ki jaan hai.

Validation — chaar tools, chaar alag kaam

Tool	Kya pakadta hai
Glue Data Quality	Known unknowns — jo galti aapko pata hai (rules likhe hue)
Data Wrangler	Unknown unknowns — jo pata hi nahi tha (graph dekhne par dikha)
Custom Lambda	Jo sirf aapka business jaanta hai (VIN match, medical codes)
CloudWatch	Galtiyaan badh toh nahi rahi? (trend)

Data Wrangler ka asli example: transcript mein isse pata chala ki purane systems mein policy number ka **format alag-alag** hai. Yeh rule se nahi milta — kyunki aapko pata hi nahi tha ki formats alag hain. **Graph dekhne par dikha**.

Processing — parallel pipelines

```

+--> Text  (Comprehend)
Claim --+--> Image (Rekognition)
+--> Audio (Transcribe)
+--> Table (SageMaker Processing)
      |
      | sab poore hue?
      |
      | DATA FUSION
```

Step Functions kyun, Lambda kyun nahi: Lambda 15 min mein khatam, Transcribe 30 min le sakta hai. Aur **sabka intezaar karna** — Step Functions khud jaanta hai ki chaaron branches पूरी hui.

Rekognition vs SageMaker Processing: Rekognition generic hai (objects, quality). Damage severity kitni hai — **yeh generic tool nahi bata sakta**, apna model chahiye.

Data Fusion — sabse mushkil hissa

Sirf paste karna nahi hai. Do problem:

1. Alignment: call mein 3:45 par customer ne kaha "peeche se thok diya". Photo #4 mein peeche ka bumper toota hai. **Yeh dono ek hi baat hain** — jodna padega.

2. Contradiction — yahi asli value hai:

Form kehta hai:	"front bumper damage"
Photo dikhata hai:	"rear damage"
Call mein kaha:	"someone hit me from behind"

Form galat hai — **ya fraud hai**. Aur yeh **sirf tab dikhta hai jab teeno saath rakho**. Alag-alag dekhoge toh miss ho jaayega. Isiliye fraud detection 43% behtar hui.

Formatting — inference parameters

Parameter	Kya karta hai
temperature	Randomness. 0 = hamesha same jawaab. Insurance mein 0-0.2 — ek claim ka jawaab har baar alag nahi ho sakta.
top_p	Kitne options mein se chune. temperature aur top_p dono mat ghumao — ek chuno.
max_tokens	Sirf output ki limit, input ki nahi

Conversation history

Model ko kuch yaad nahi rehta. Har request mein पूरी history dobara bhejni padti hai. "Memory" aapko banani padti hai.

Teen tarike: **sliding window** (aakhri 10 messages), **summarization** (purani baat ka nichod), **hybrid** (summary + haal ke messages) — hybrid sabse common.

Enhancement — validation se alag

	Kya karta hai
Validation	Kharab data rok deta hai
Enhancement	Theek-thaak data ko behtar banata hai

Bedrock se hi safai: chhota model pehle text saaf kare (abbreviations kholo, grammar theek karo), phir main model asli kaam kare.

Content enrichment — sabse zyada value:

Claim mein hai: "2019 Honda Civic, rear bumper damage"
+ vehicle specs
+ repair cost benchmark (is area mein average \$850)
+ is model ki claim history

Ab model bol sakta hai: **"\$4,500 ka estimate benchmark se 5x zyada hai — flag karo."** Bina enrichment ke wo yeh nahi keh sakta, kyunki usko pata hi nahi ki normal kya hai.

Task 1.4 — Vector Stores

Use case

Law firm — 30 countries, 1.5 crore documents. Attorneys hafte mein 15 ghante research mein lagate the.

Chaar stores, chaar wajah

Store	Kyun
Knowledge Bases	Public docs — fully managed, kam kaam
OpenSearch	Internal opinions — hybrid search + security boundaries
RDS + S3	Case metadata — SQL joins jo vector store nahi kar sakta
DynamoDB	Live regulations — sub-ms lookups

Hybrid search — legal ka asli problem

Semantic search: "contract todna" -> "breach of contract" MIL GAYA
Keyword search: "Section 230" -> bilkul wahi MILA

Legal mein dono chahiye. Attorney "Section 230" dhoondh raha hai toh usse *milta-julta* nahi chahiye — **wahi** chahiye. Semantic search "Section 220" bhi de dega, kyunki matlab milta-julta hai.

RDS kis liye — aapka sawaal

Aapne poocha tha ki metadata vectors ka hoga ya original ka. **Do alag cheezein hain:**

	Kya rakhta hai	Kaam
Vector store metadata	Chunk-level, thoda sa	Search ke andar filter
RDS	Document-level, bahut zyada	SQL joins, relationships

"Konse cases Judge Smith ne likhe aur 5 se zyada baar cite hue?" — **yeh join hai. Vector store yeh kar hi nahi sakta.**

Metadata itna important kyun

Vector search bas **matlab** dekhta hai. Ek overturned case aur ek valid case — **dono ka embedding lagbhag same hoga**, kyunki matlab toh wahi hai. Sirf metadata batata hai ki ek ab kaam ka nahi.

Hierarchical tagging — ped ki tarah:

Practice Area
Litigation
Commercial
Contract Disputes

Tag inheritance: "Litigation" search karo, neeche ke saare mil jaate hain.

Cross-reference metadata — legal ki jaan: aapko perfect case mila, par kisi ne use 2015 mein overturn kar diya. **Vector search yeh nahi bata sakta.** Relationship graph batata hai.

Performance — 1.5 crore documents par sub-second

Technique	Kya deta hai
Sharding	Practice area se shard, phir time se sub-shard. Contract query → sirf contract shards touch hote hain.
Multi-index	Har index ki apni tuning — tax law aur IP law ki bhasha alag hai
Hierarchical	Coarse-to-fine — 15M ki jagah 500K search hua
Quantization	Memory bachi, speed badhi, accuracy thodi giri
Caching	Common queries pehle se tayaar

Maintenance

Idea: vector store ek **photo** hai, duniya **video** hai.

Tarika	Kab
Incremental	Roz ke changes — sirf jo badla
Real-time	Turant chahiye (S3 events, DynamoDB Streams)
Scheduled full rebuild	Naya embedding model aaya, ya fragmentation

Incremental update mein teen operations hain: naya → add, badla → **purane chunks delete karo phir naye daalo**, delete hua → remove. Doosra step log bhool jaate hain aur stale chunks reh jaate hain.

Aur **full rebuild kyun zaroori:** naya embedding model aaya toh purane aur naye embeddings **compare hi nahi ho sakte** — alag space mein hain.

Cache invalidation mat bhoolein: naya precedent aaya, par purana cached jawaab attorney ko mil raha hai.

Task 1.5 — Retrieval Design

1.4 vs 1.5

Task	Focus
1.4	Vector store kahan rakho, kaise sambhalo (infrastructure)
1.5	Retrieval kitna achha hai — recall aur relevance (quality)

Chunking — legal mein sabse mushkil

"The Party may terminate this Agreement, provided that Section 5.1 conditions are met."

Beech mein kaat do → chunk kehta hai "Party may terminate". **Conditions gayab. Matlab ulta ho gaya.** Legal mein galat chunking = galat legal advice.

Hierarchical chunking — sabse powerful:

PARENT: Article 5 - Termination (2000 words)
+-- CHILD: 5.1 Conditions (300 words)
+-- CHILD: 5.2 Notice (250 words)

Search chhote child par hota hai (zyada focused, better match), **par model ko parent bheja jaata hai** (poora context). Best of both.

Embeddings

Titan Text Embeddings: 1,536 dimensions — yeh number yaad rakhiye.

Binary embeddings: har number 1 bit mein — **32x kam storage**, thodi accuracy kam.

Domain fit dimensions se zyada important hai. General model legal bhasha nahi jaanta — "consideration" ka contract law mein bilkul alag matlab hai. Transcript mein general content par Titan use hua, specialized practice areas par **custom fine-tuned embedding models**.

Reranker — do-step process

Step 1: vector search -> top 100 (tez, mota)
Step 2: reranker -> top 10 (dheema, baareek)

Reranker zyada accurate kyun hai: vector search mein query aur document **alag-alag** embed hote hain, phir compare hote hain. Model ne dono ko saath mein kabhi nahi dekha. **Reranker dono ek saath padhta hai.**

Toh sirf reranker kyun nahi? Wo slow hai — 1.5 crore documents par nahi chala sakte.

Query handling — yahan sabse zyada naya hai

Problem 1 — query bahut chhota:

"non-compete enforceability" (do shabd)

Query expansion (Bedrock se):
"non-compete enforceability, restrictive covenants, employee mobility, trade secret protection, reasonable restraint of trade"

+23% recall improvement. Attorney ne yeh terms nahi likhe, par relevant documents mein yahi shabd hain.

Problem 2 — query bahut complex: "Kya California mein non-compete enforce hoga agar employee Texas mein hai aur trade secrets bhi involve hain?" — **yeh ek sawaal nahi, teen hain.**

Ek saath search karoge toh vector **teen concepts ka average** ban jaayega, aur kisi ka sahi jawaab nahi milega. Isliye Lambda se **decompose** karo, Step Functions se **parallel** chalao — 40% time reduction.

MCP — model aur tools ke beech

Analogy: USB jaisa. Pehle har device ka apna cable tha; USB ne ek standard bana diya. MCP wahi karta hai — ek baar MCP server banao, koi bhi MCP-compatible model use kar le.

Citations

Har jawaab mein source chahiye. Isliye chunks ke saath **source metadata** bhi jaata hai, aur **har vector ka original document se mapping** rakhna padta hai — warna citation de hi nahi sakte.

Task 1.6 — Prompt Engineering aur Governance

Core idea

Prompt ek code artifact hai. Regulated industry mein prompt badalna production mein deploy karne jaisa hai — review, approval, audit trail sab chahiye.

Sochiye — ek line badal do prompt mein, aur AI 5 crore customers ko galat financial advice dene lage.

Teen layers of control

Prompt Management	-> "aisa behave karo"	(instruct)
Guardrails	-> "yeh mat kaho"	(block)
JSON Schema	-> "is shakal mein jawaab do"	(structure)

Guardrails ki severity levels: chaar categories (hate, insults, sexual, violence), har ek LOW/MED/HIGH. **HIGH matlab sirf sabse severe block hoga; LOW matlab halka bhi block ho jaayega** — yani LOW zyada strict hai.

JSON Schema ka chalak use: `disclaimer` ko **required field** bana do. Ab model use chhod hi nahi sakta. **Compliance ko schema mein daal do, phir wo optional nahi rehta.**

Aur sabse zaroori: Guardrails **model se alag** hai aur **prompt ke bahar** hai. Isliye prompt badal kar ya prompt injection se use bypass nahi kiya ja sakta.

Interactive systems

Clarification loop: "Mera transfer nahi hua" — konsa transfer? Model guess karega toh galat jawaab dega. **Poocho, guess mat karo.**

Comprehend custom classification — 200+ intents. Model se kyun nahi poochte? Kyunki Comprehend **sasta, tez, aur deterministic** hai — same input, same output. Banking mein zaroori.

DynamoDB TTL ek compliance feature hai, cleanup nahi. 90 days ke baad data apne aap delete. Retention policy **automatically enforce** hoti hai — koi script nahi, koi bhoolne ka chance nahi.

Governance — three-tier approval

1. Banking expert	-> content sahi hai?
2. Compliance reviewer	-> regulation follow hoti hai?
3. System admin	-> deploy karo

Teen alag log kyun: jo likhta hai wo apni galti nahi dekh paata. Aur compliance ek alag skill hai — banking expert regulation nahi jaanta.

Service	Kya record karta hai
CloudTrail	Kisne, kab, kaunsa prompt — metadata (7 saal retention)
CloudWatch Logs	Kya bheja, kya aaya — content

Quality assurance

Mechanism	Kab chalta hai
Output validation	Har request par, real-time
Edge case testing	Deploy se pehle (joint accounts, international transfers)
CloudWatch KPIs	Production mein, continuously
Regression testing	Har naye version par

Escalation rate sabse chalak metric hai. Achanak badh gaya matlab AI kaam nahi kar pa raha — **par error rate normal rahega.** Sirf yahi metric batayega.

Regression testing — yeh concept exam mein aata hai:

Test case	v7	v8	
Joint account	62%	-> 94%	sudhra
Intl transfer	91%	-> 73%	BIGDA
Balance inquiry	98%	-> 97%	theek hai

Aap v8 banate ho joint accounts theek karne ke liye. **Wo theek ho jaata hai, par international transfers kharab ho jaate hain — aur pata hi nahi chalta.** Bina regression testing ke aap ek problem theek karke doosri banate rahoge.

Chain-of-Thought

Bina CoT: "Aapko Rs 5,32,000 milega." (galat, par confident)

- CoT ke saath:
1. Principal Rs 5,00,000, rate 7%
 2. 1 saal ka interest = Rs 35,000
 3. Early withdrawal penalty = 1% reduction
 4. Adjusted rate 6% -> interest Rs 30,000
 5. Premium tier: penalty 50% waived
 6. Total = Rs 5,32,500

Kyun kaam karta hai: model ek token ke baad agla token banata hai. Agar wo pehle reasoning likhta hai, toh **wo reasoning uske apne context mein aa jaati hai** — aur agla step us par bana hota hai. Seedha jawaab maangoge toh sab kuch ek jump mein karna padega.

Bonus: reasoning dekh kar pata chalta hai galti kahan hui. Par customer ko dikhana zaroori nahi — post-processing mein sirf final answer nikaal lo.

Prompt Flows

Ek prompt kaafi nahi jab kaam multi-step ho (mortgage prequalification). Flows ek **graph** hai — Prompt, Condition, Lambda, Knowledge Base nodes.

Conditional branching: credit score 750+ → premium flow, 650-749 → standard, <650 → improvement guidance. Aur **confidence-based:** <0.7 → human agent.

Post-processing mein disclaimer jodna chalak hai — model par bharosa nahi karna padta ki wo yaad rakhega. **System jodta hai, hamesha.**

Reusable components ka compliance fayda: "APR" ka explanation mortgage, credit card aur personal loan teeno mein chahiye. Ek component banao — regulator ne wording badalne ko kaha toh **ek jagah badlo, sab jagah lag gaya.**

Domain 1 — Yaad rakhne layak dus baatein

#	Baat
1	Anycast Marie ko dhoondhta nahi — har router apne se chhota raasta chunta hai
2	GA sirf edge par shuru hota hai; do IPs redundancy hain, region nahi
3	Model kabhi search nahi karta. Lambda retrieve karta hai, phir prompt mein daalta hai
4	Metadata ingestion ke waqt daalo; embedding model dono taraf same
5	RAG facts deta hai, fine-tuning tareeka — fine-tuning cutoff theek nahi karta
6	AppConfig ek settings file hai, jagah nahi — Router usse padhta hai
7	Fusion mein contradiction pakadna hi asli value hai (fraud detection)
8	Vector store metadata (chunk-level, filter) aur RDS (document-level, joins) alag hain
9	Hierarchical chunking — chhote se search, bada bhejo
10	Prompt = code. Version, review, approve, audit — sab chahiye